

AI ASSISTED CODING -13.3

NAME: T. SAI VARUN

HTK. NO: 2403A510C0

BATCH NO:05

TASK #1 – REMOVE REPETITION

AI Prompt:

Refactor redundant code using dictionary-based dispatch or modular functions.

Refactored Code:

```
Ai_13.py > calculate_area
1  def rectangle_area(x, y):
2      return x * y
3
4  def square_area(x, _):
5      return x * x
6
7  def circle_area(x, _):
8      return 3.14 * x * x
9
10 area_dispatch = {
11     "rectangle": rectangle_area,
12     "square": square_area,
13     "circle": circle_area
14 }
15
16 def calculate_area(shape, x, y=0):
17     if shape in area_dispatch:
18         return area_dispatch[shape](x, y)
19     else:
20         raise ValueError(f"Unknown shape: {shape}")
21 print(calculate_area("rectangle", 5, 3)) # Output: 15
22 print(calculate_area("square", 4))      # Output: 16
23 print(calculate_area("circle", 2))      # Output: 12.56
```

AI ASSISTED CODING -13.3

Output:



The screenshot shows a Windows PowerShell terminal window with a dark background. The title bar at the top includes tabs for 'PROBLEMS', 'OUTPUT', 'TERMINAL' (which is active), 'DEBUG CONSOLE', and 'PORTS'. On the right side of the title bar, there are icons for a file explorer, a plus sign, a window icon, a trash can, and a close button. The terminal content shows a command prompt 'PS C:\Users\saiva\OneDrive\Desktop\Ai>' followed by the execution of a Python script: `& C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/Ai/Ai_13.py`. The output of the script is displayed on the following lines: '15', '16', and '12.56'. The prompt returns to 'PS C:\Users\saiva\OneDrive\Desktop\Ai>'.

```
PS C:\Users\saiva\OneDrive\Desktop\Ai> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/Ai/Ai_13.py
15
16
12.56
PS C:\Users\saiva\OneDrive\Desktop\Ai>
```

Observation:

Dictionary-based dispatch removes repetitive conditions and improves modularity.

TASK #2 – ERROR HANDLING IN LEGACY CODE

AI Prompt:

Refactor legacy function to use with `open()` and proper error handling.

Refactored Code:

```
Ai_13.py X
Ai_13.py > ...
1 def read_file(filename):
2     try:
3         with open(filename, "r") as f:
4             return f.read()
5     except FileNotFoundError:
6         return f"Error: The file '{filename}' was not found."
7     except Exception as e:
8         return f"An error occurred: {e}"
9
10 print(read_file("example.txt"))
11
```

Output:



The screenshot shows a Windows PowerShell terminal window with the following content:

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
PS C:\Users\saiva\OneDrive\Desktop\Ai> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/Ai/Ai_13.py
Error: The file 'example.txt' was not found.
PS C:\Users\saiva\OneDrive\Desktop\Ai>

```

Observation:

Using with `open()` ensures automatic file closing and `try-except` makes it robust.

TASK #3 – COMPLEX REFACTORING

AI Prompt:

Improve class readability and modularity with meaningful variable names, docstrings, and list for marks.

AI ASSISTED CODING -13.3

Refactored Code:

```
Ai_13.py > ...
1  class Student:
2      """
3      Represents a student with a name, age, and a list of marks.
4      """
5
6      def __init__(self, name: str, age: int, marks: list[int]):
7          self.name = name
8          self.age = age
9          self.marks = marks # Store marks as a list for flexibility
10
11     def display_details(self):
12         """Print the student's name and age in a readable format."""
13         print(f"Name: {self.name}, Age: {self.age}")
14
15     def total_marks(self) -> int:
16         """Return the total of all marks."""
17         return sum(self.marks)
18
19     # Example usage:
20     student = Student("Alice", 20, [85, 90, 78])
21     student.display_details()
22     print("Total Marks:", student.total_marks())
23
```

Output:

```
PROBLEMS  OUTPUT  TERMINAL  DEBUG CONSOLE  PORTS
PS C:\Users\saiva\OneDrive\Desktop\Ai> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/Ai/Ai_13.py
• Name: Alice, Age: 20
  Total Marks: 253
○ PS C:\Users\saiva\OneDrive\Desktop\Ai> |
```

Observation:

Improved variable names, added docstrings, and used sum(self.marks) for scalability.

TASK #4 – INEFFICIENT LOOP REFACTORING

AI Prompt:

Refactor inefficient loop using list comprehension.

AI ASSISTED CODING -13.3

Refactored Code:

 Ai_13.py

×

```
1  nums = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
2  squares = [i * i for i in nums]
3  print(squares)
4
```

Output:

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS

powerShell

+

⌵

🗑

⋮

|

🔍

×

PS C:\Users\saiva\OneDrive\Desktop\Ai> & C:\Users\saiva\AppData\Local\Programs\Python\Python313\python.exe c:/Users/saiva/OneDrive/Desktop/Ai/Ai_13.py

• [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

○ PS C:\Users\saiva\OneDrive\Desktop\Ai>

Observation:

List comprehension makes the code concise, readable, and slightly faster.

SUMMARY TABLE:

Task	Key Improvement	Technique Used
1	Removed repetitive if-elif	Dictionary dispatch + functions
2	Safe file handling	with open() + try-except
3	Improved readability & modularity	Docstrings, lists, formatting
4	Simplified loop	List comprehension